

# Vitamin Helix

Infrastructure & Deployment Reference

## Architecture Overview

Vitamin Helix runs on a self-hosted local network setup within the Vitamin Dubai office. The system is composed of three physical nodes: a development laptop, a Mini-PC server, and a Synology NAS — all on the same LAN.

| Node                  | Role                      | OS / Software                      |
|-----------------------|---------------------------|------------------------------------|
| Windows Laptop        | Development & Testing     | Windows / VS Code / Python / Flask |
| Mini-PC (Server Room) | Production App Server     | Linux / Flask / PostgreSQL 18      |
| Synology NAS          | File Storage & DB Backups | Synology DSM / SMB / VPN           |

## Network Flow

[Windows Laptop] → git push → [GitHub]  
↓ git pull (SSH)  
[Mini-PC / Linux – Server Room]  
Helix Flask App | PostgreSQL  
↓ pg\_dump (cron, daily)  
[Synology NAS – Admin/Database/]  
VPN tunnel for remote file access  
Synology DSM for NAS management

## Remote Access

|              |                                                                                        |
|--------------|----------------------------------------------------------------------------------------|
| Mini-PC      | AnyDesk installed — use for remote desktop access if maintenance needed outside office |
| Mini-PC      | SSH available on LAN for fast terminal access and deployments                          |
| Synology NAS | Managed via Synology DSM (browser-based). No AnyDesk needed — DSM handles updates      |
| NAS Files    | VPN tunnel for secure remote access to files from outside the network                  |

## Deployment Workflow

Every update follows this flow from dev laptop to production Mini-PC:

### 1. Develop & test

Write code on the Windows laptop. Test locally with Flask dev server.

### 2. Commit & tag

```
git commit -m "description"
git tag v1.x
git push
```

### 3. SSH into Mini-PC

```
ssh ezekiell@
```

### 4. Pull & restart

```
cd ~/project-tracker
git pull
sudo systemctl restart helix
```

### 5. Schema changes

If any ALTER TABLE migration scripts are needed, run them manually after pull:

```
python add_example_column.py
```

*Tip: One-liner deploy from your laptop: ssh ezekiell@ "cd ~/project-tracker && git pull && sudo systemctl restart helix"*

## Backup Strategy

|                        |                                                                              |
|------------------------|------------------------------------------------------------------------------|
| <b>Daily snapshot</b>  | pg_dump runs via cron on Mini-PC. Output saved to NAS: Admin/Database/daily/ |
| <b>Weekly snapshot</b> | Full pg_dump every Sunday saved to NAS: Admin/Database/weekly/               |
| <b>Retention</b>       | Keep last 30 daily dumps. Cron auto-deletes older files.                     |
| <b>Before deploy</b>   | Manual pg_dump recommended before any major update (save to NAS)             |
| <b>NAS location</b>    | Synology NAS → Admin / Database /                                            |

### Daily cron job (Mini-PC)

```
0 2 * * * pg_dump -U postgres -d project_tracker -F c -f
/mnt/nas/Admin/Database/daily/$(date +%Y-%m-%d).dump
0 2 * * 0 pg_dump -U postgres -d project_tracker -F c -f
/mnt/nas/Admin/Database/weekly/$(date +%Y-%m-%d).dump
```

## Initial Migration to Mini-PC (Friday)

One-time setup to move from dev laptop to the Mini-PC production server:

1. Install Linux on Mini-PC
2. Install PostgreSQL 18, Python 3.x, pip, git
3. Clone repo: git clone
4. Install dependencies: pip install -r requirements.txt
5. On dev laptop – export DB: pg\_dump -U postgres -d project\_tracker -F c -f helix\_launch.dump
6. Copy helix\_launch.dump to Mini-PC (USB or via NAS share)
7. On Mini-PC – create DB: createdb -U postgres project\_tracker
8. Restore: pg\_restore -U postgres -d project\_tracker -F c helix\_launch.dump
9. Set up .env / config with correct DB credentials
10. Run app and verify everything works
11. Set up systemd service so app starts on boot
12. Install AnyDesk
13. Mount NAS share and configure backup cron jobs

## Health & Monitoring

|                        |                                                                              |
|------------------------|------------------------------------------------------------------------------|
| <b>Health endpoint</b> | Add /ping route to Helix returning HTTP 200. Used for uptime checks.         |
| <b>Uptime check</b>    | Cron on Mini-PC checks /ping every 5 mins. Sends email/notification if down. |
| <b>Disk alert</b>      | Monitor Mini-PC disk usage — alert if above 80% capacity.                    |
| <b>DB size</b>         | Periodic check on project_tracker DB size to anticipate storage needs.       |

## Security Notes

|                         |                                                                            |
|-------------------------|----------------------------------------------------------------------------|
| <b>Network exposure</b> | Helix bound to LAN only — not exposed to the public internet               |
| <b>Remote access</b>    | VPN required to access Helix or NAS from outside the office network        |
| <b>PostgreSQL</b>       | Postgres listens on localhost only on Mini-PC. Strong password set.        |
| <b>SSH</b>              | SSH key-based auth recommended. Disable password login on Mini-PC.         |
| <b>Git</b>              | Repo on GitHub (private). No secrets committed — use .env for credentials. |

## Rollback Procedure

If a deployment breaks something:

```
# Code rollback – revert to previous git tag
git checkout v1.x
sudo systemctl restart helix

# Database rollback – restore from last NAS snapshot
pg_restore -U postgres -d project_tracker -F c
/mnt/nas/Admin/Database/daily/YYYY-MM-DD.dump
```